Question-Driven Tabular Data Preparation Using A Self-Correcting Framework

Feng Luo[1], Hui Luo[2], Zhifeng Bao[3], J. Shane Culpepper[3], Xiaoli Wang[4]
[1]RMIT University, [2]University of Wollongong, [3]The University of Queensland, [4]Xiamen University

ABSTRACT

In this paper, we study *Question-Driven Tabular Data Preparation*, which aims to construct a pipeline that identifies and prepares the relevant tables and establishes their relationships to answer a question. This task is important in enterprise analytics, with 76% reporting preparation needs. We highlight three key challenges in solving this problem: *Reliable Relevant Table Identification*, *Pipeline Synthesis under Increasing Complexity*, and *Error Mitigation*. Existing methods provide limited support to address all of these challenges. To address this gap, we propose PrepWeaver, a self-correcting preparation framework consisting of: (1) *Table Discovery*, which improves offline metadata and identifies relevant tables through joint question coverage and bridge-table expansion; (2) *Clique-Based Pipeline Synthesis*, which synthesizes the preparation pipeline through maximum-weight clique search over a weighted complete multipartite graph; and (3) *Diagnosis-Guided Self-Correction*, which uses a learned diagnostic policy and a correction tree to mitigate error propagation. Experiments on three benchmarks show that PrepWeaver improves Preparation Correctness by 47% on average at only about 49% of the monetary cost of the lowest-cost API-based LLM baseline. Notably, Self-Correction further improves Preparation Correctness by 20%.

PVLDB Reference Format:
Feng Luo[1], Hui Luo[2], Zhifeng Bao[3], J. Shane Culpepper[3], Xiaoli Wang[4]. Question-Driven Tabular Data Preparation Using A Self-Correcting Framework. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/JrJessyLuo/Prep-Weaver.

1 INTRODUCTION

Towards Realistic Tabular Data Preparation. Table collections are widely used in real-world enterprise applications such as business intelligence [29] and financial analytics [15], where users frequently pose natural language questions to obtain the information needed for analysis and decision making. Answering such questions can be challenging since the required information may be distributed across multiple tables that must first be identified, and these tables often lack explicit relationships such as joinability [17, 24, 49] and may also be non-relational [29, 31]. Consequently, answering such questions often requires data preparation first. Around 76% of enterprise users have reported such preparation needs [5]. In this paper, we study *Question-Driven Tabular Data Preparation*: Given a natural language question and a table collection, construct a data preparation pipeline that identifies and prepares the relevant tables and establish their relationships required to answer the question.

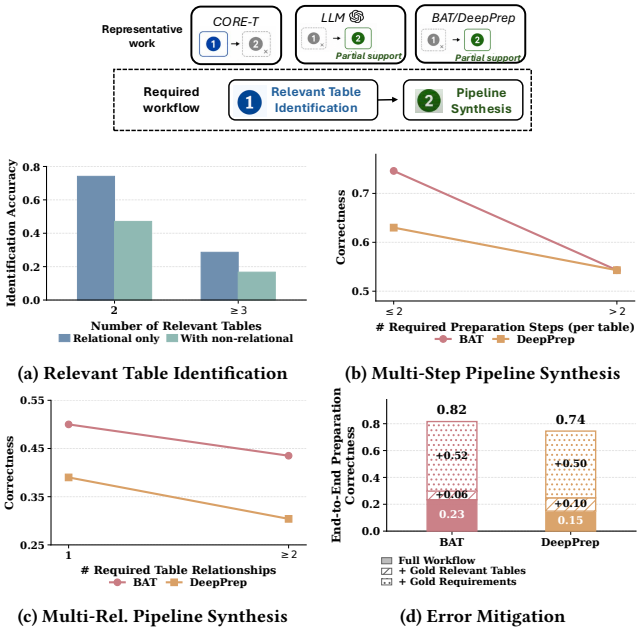


Figure 1: Research gap and empirical analysis.

Research Gap. Solving this problem requires an end-to-end workflow with two stages: (1) *Relevant Table Identification*, which identifies all relevant tables from the table collection; and (2) *Pipeline Synthesis*, which first determines per-table requirements (i.e., what each relevant table should be prepared as) and the relationships among the resulting tables, and then synthesizes a preparation pipeline to realize them. However, *existing methods [19, 21, 39, 42] provide only partial coverage of this workflow*, as illustrated at the top of Figure 1, which we will discuss shortly in §2.2. Specifically, CORE-T [42] addresses Stage (1), BAT [21] and DeepPrep [19] primarily synthesize pipelines for per-table requirement realization, partially covering Stage (2). Using an LLM [39] can also potentially determine the requirements and relationships needed in Stage (2). This gap motivates us to investigate whether this problem can be effectively addressed by composing existing methods into such an end-to-end workflow. Our initial empirical analysis[1] shows that simply composing these methods remains insufficient, revealing three key challenges that must be addressed to provide an effective end-to-end solution:

[1]We conducted an analysis on the dataset Synth-Bird (§6.1).

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

(C1) **Reliable Relevant Table Identification.** Figure 1(a) demonstrates the accuracy of identifying relevant tables using CORE-T with respect to the number of relevant tables and whether non-relational tables are involved among them. Accuracy drops by about 63% as the number of relevant tables required increases and is lower (39%) when non-relational tables are involved. *Thus, reliably identifying all relevant tables for a question remains difficult when the information required is distributed across multiple tables, particularly when the tables are non-relational.*

(C2) **Pipeline Synthesis under Increasing Complexity.** Given the ground-truth relevant tables and LLM-derived requirements, Figures 1(b) and (c) analyze pipelines synthesized using BAT and DeepPrep under two different conditions of increasing complexity: the number of preparation steps required per table and the number of required table relationships. Figure 1(b) shows that the correctness of preparing each table decreases by about 20% as the number of required preparation steps increases, while Figure 1(c) shows that relationship correctness decreases by about 18% with more required table relationships. *Thus, effectively synthesizing pipelines becomes more difficult as more preparation steps and table relationships are required.*

(C3) **Error Mitigation.** Figure 1(d) evaluates the composed workflow by progressively replacing the identified relevant tables and the derived requirements with their ground-truth counterparts. Errors in relevant table identification lead to an average relative drop of 30% in end-to-end preparation correctness, while errors in pipeline synthesis lead to a further average relative drop of 65%. *Thus, upstream errors can be propagated through the workflow and degrade end-to-end preparation correctness.*

Our Solution. To address these challenges, we introduce PrepWeaver, a self-correcting preparation framework as an effective end-to-end solution. As shown in Figure 2, PrepWeaver first constructs a preparation workflow that identifies all relevant tables and then synthesizes a complex pipeline. Building on this workflow, self-correction is applied to minimize error propagation and improve overall effectiveness.

Table Discovery (§3). To address **C1**, we first synthesize high-quality table metadata for non-relational tables. Then, we select candidate table sets through joint coverage of question keyphrases and further augment them with bridge tables, thereby improving the identification of more relevant tables.

Clique-Based Pipeline Synthesis (§4). To address **C2**, we formulate pipeline synthesis as a maximum-weight clique problem over a weighted complete multipartite graph. First, we use target-guided pruning to generate candidate multi-step preparation sequences as vertices, with vertex weights measuring table correctness and edge weights measuring relationship correctness. We then perform maximum-weight clique search to jointly select the candidate preparation sequences, thereby improving pipeline synthesis as more preparation steps and table relationships are required.

Diagnosis-Guided Self-Correction (§5). To address **C3**, we first introduce a self-correction mechanism. Building on this mechanism, we iteratively diagnose workflow execution state using a learned diagnostic policy and apply any corresponding corrections using a correction tree, thereby further improving preparation correctness.

In summary, this work makes the following contributions:

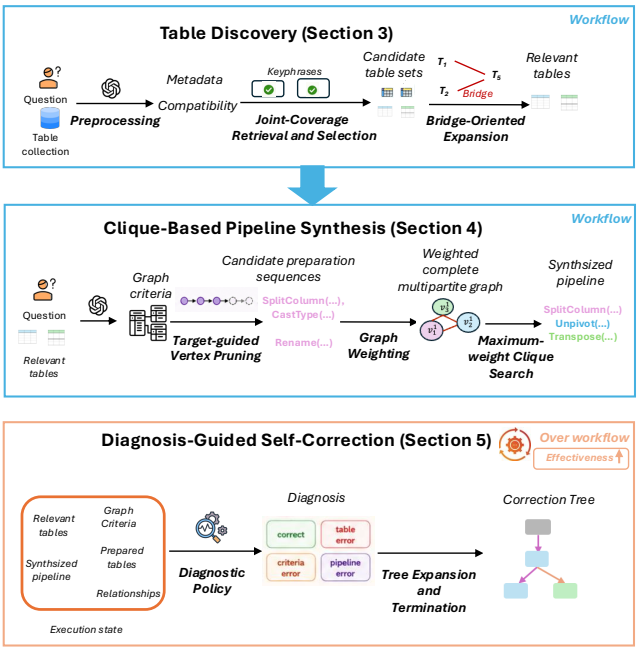


Figure 2: Overview of PrepWeaver.

- We identify three key challenges in Question-Driven Tabular Data Preparation: *Reliable Relevant Table Identification*, *Pipeline Synthesis under Increasing Complexity*, and *Error Mitigation*. To address each of these challenges, we propose PrepWeaver, a self-correcting preparation framework that constructs a preparation workflow while mitigating error propagation throughout the workflow, thus providing an effective end-to-end solution.
- We develop *Table Discovery*, which improves table metadata and selects relevant tables through joint question coverage and bridge-table expansion, improving relevant table identification by 17%.
- We develop *Clique-Based Pipeline Synthesis*, which synthesizes the preparation pipeline using a maximum-weight clique search over a weighted complete multipartite graph, which can improve preparation correctness by 15%.
- We develop *Diagnosis-Guided Self-Correction*, which uses a learned diagnostic policy and a correction tree to minimize error propagation, which can improve preparation correctness by 20%.
- We conduct comprehensive experiments to evaluate the effectiveness, robustness, and efficiency of PrepWeaver using three benchmarks. PrepWeaver improves Preparation Correctness by 47% over best-performing baselines, while incurring only about 49% of the monetary cost of other API-based LLM methods. (§6)

2 PROBLEM AND LITERATURE REVIEW

2.1 Problem Definition

Table Collection. Let T be a flat table with columns and rows. Let $\mathcal{T} = \{T_1, \ldots, T_N\}$ denote a collection of tables. Unlike table collections from relational databases, we focus on table collections whose tables may not conform to the relational format (i.e., rows represent records, columns represent attributes, and values within each column are homogeneous) [31] and lack explicit table relationships.

Operation	Description
Structural Transformations	
Transpose	Swaps rows and columns.
Pivot	Reshapes long-format data into wide format.
Unpivot	Reshapes wide-format data into variable–value pairs.
WideToLong	Reshapes repeated wide-format columns into long format using header patterns.
Schema Transformations	
SplitColumn	Splits one column into multiple columns.
Concatenate	Combines multiple columns into one column.
Rename	Replaces column headers with specified names.
Value Normalizations	
StandardizeString	Normalizes string values in a column.
StandardizeDatetime	Converts values to a target datetime format.
CastType	Converts values to a target data type.

Table 1: Overview of the supported preparation operations.

Data Preparation Operations. Following [19, 29], we consider a set of data preparation operations O. As summarized in Table 1, we organize these operations into three classes according to the primary table elements they modify: *structural transformations*, which reorganize values across rows and columns and may change both the columns and rows; *schema transformations*, which modify column headers and may also modify the corresponding values in each row; and *value normalizations*, which modify column values.

Data Preparation Pipeline. A data preparation pipeline is an ordered sequence of parameterized preparation steps, $P = (p_1, \ldots, p_L)$. Each step $p_i = (o_i, \theta_i, X_i)$ applies an operation $o_i \in O$, instantiated with operation-specific parameters θ_i, to an input table X_i and produces an output table $Y_i = o_{i\theta_i}(X_i)$. The input X_i is either a raw table in $\mathcal{T}$ or an intermediate table Y_j produced by an earlier step p_j, where $j < i$. Executing P in order therefore produces the intermediate tables $\{Y_1, \ldots, Y_L\}$.

Joinability Specification. In this paper, we focus on joinability [17, 24, 27, 49] as a representative table relationship. Two tables are considered joinable if they contain corresponding columns whose values overlap or are semantically similar. For a set of tables, a *joinability specification* records which table pairs are joinable and the corresponding columns through which they can be joined.

Definition 1 (Question-Driven Tabular Data Preparation). *Given a natural language question q and a table collection $\mathcal{T}$, the goal is to construct a data preparation pipeline P that produces a set of prepared tables $\mathcal{S}$ ($|\mathcal{S}| \geq 2$), together with a joinability specification $\mathcal{K}_\mathcal{S}$ for these tables. Each table in $\mathcal{S}$ is an intermediate table produced by P. The tables in $\mathcal{S}$ can be integrated according to $\mathcal{K}_\mathcal{S}$ to answer q.*

2.2 Literature Review

Relevant Table Identification. Relevant table identification aims to identify the tables required for answering a question from a table collection. As our setting focuses on questions that require multiple relevant tables, we mainly consider existing work on multi-table retrieval. Existing methods differ in whether they explicitly recover bridge tables [13], i.e., intermediate tables needed to connect multiple query-relevant tables. Some methods [9, 13, 33, 46, 48] retrieve tables mainly based on their semantic relevance to the question, without explicitly recovering such bridge tables. Other methods [6, 12, 42] additionally exploit table–table compatibility to recover bridge tables. Among them, CORE-T [42] is a recent state-of-the-art method that first retrieves query-relevant tables and then augments them through compatibility-guided expansion.

However, these methods focus only on identifying the relevant tables, and cannot synthesize the preparation pipeline, thus providing only partial support for our setting.

Data Preparation Pipeline Synthesis. Existing specification-guided pipeline synthesis methods [19, 21, 22] generate executable preparation pipelines for given input tables under explicit preparation specifications. For example, BAT [21] and DeepPrep [19] specify the target-table metadata and use LLMs to synthesize the operations to produce a table satisfying target. While Text-to-Pipeline [22] specifies natural-language instructions that describe the preparation procedure and grounds it into a pipeline using LLMs.

Despite their effectiveness, these methods support only part of the workflow required in our setting. First, they assume that the relevant tables to be prepared are already given, and therefore do not consider relevant table identification. Second, they require explicit preparation specifications for pipeline synthesis, while our setting provides only the question and must infer these specifications.

Self-Correction. Self-correction [23, 34, 41] aims to improve the reliability of LLM's outputs by iteratively identifying and correcting errors using feedback during inference. Existing methods mainly differ in the feedback's source. Some methods [34, 41] rely on LLM-generated feedback, where the model critiques its previous output and uses the resulting feedback to refine the next output. Other methods [23] rely on externally grounded feedback, where feedback is obtained from the execution results of external tools.

However, existing self-correction methods are mainly designed to directly refine an erroneous output of LLM, rather than correct errors within a multi-stage workflow, and thus cannot be applied to address error propagation through the preparation workflow.

Other Loosely Related Work. They broadly fall into the following two categories. *Query-agnostic data preparation methods* [10, 29, 37, 50] prepare tables under generic quality objectives without conditioning on a specific question. For *other question-driven settings*, some work [18, 30] studies question-aware preparation for a given single table, rather than multiple relevant tables. Query-driven table integration [44] focuses on enabling join relationships under limited three operations, rather than preparing all relevant tables and constructing relationships with a broader set of operations. Other work [8, 36] focuses on generating programs (e.g., Python code) to answer questions over table collections, rather than explicitly generate pipelines to prepare tables and their relationships.

3 TABLE DISCOVERY

To address our problem, the first step is to reliably identify all relevant tables from the collection. In this section, we first analyze why the SOTA table identification method CORE-T [42] remains ineffective, which motivates our solution design (§3.1). We then present our detailed implementation (§3.2).

3.1 Limitations and Design Overview

CORE-T first constructs offline signals including table metadata and pairwise table–table compatibility. It then performs Table Retrieval and Table Selection to identify tables directly relevant to the question based on query–table relevance computed from the metadata, and applies Table Adjustment to further augment the selected tables through compatibility-driven greedy expansion, potentially recovering bridge tables [13].

Key Issues. However, it remains limited when involving non-relational tables or more relevant tables, mainly due to two issues.

Issue 1: Incomplete metadata for non-relational tables. When non-relational tables are involved, table metadata inferred from column headers and sampled rows may omit the attributes that are not available from these inputs.

Issue 2: Unreliable online identification of more relevant tables. Even without non-relational relevant tables, identifying more relevant tables, including bridge tables, remains unreliable because (i) Table Selection assesses table relevance against the whole question, which can miss tables relevant to only part of the question, and (ii) compatibility-driven greedy expansion can introduce spurious candidates that are highly compatible with individual tables but irrelevant to the question, especially in realistic collections with many plausible compatibility links (avg. 4.1) [11].

The left side of Figure 3 illustrates these issues.

Example 1. For Issue 1, the inferred metadata of the non-relational relevant tables T_1 and T_2 is incomplete: SegmentID is missing from T_1, while SegmentName is missing from T_2.
For Issue 2, the table providing SegmentName matches only part of the question and therefore receives a low whole-question relevance score, making it easy to be missed. Moreover, compatibility-driven greedy expansion can favor spurious tables with high pairwise compatibility (e.g., 0.87*) over the truly needed bridge table, whose compatibility with the selected tables is lower (e.g.,* 0.43 *and* 0.41*).*

Reliable Table Discovery. To address these issues of CORE-T, we propose Table Discovery to effectively identify relevant tables (see the right side of Figure 3).

During the offline phase, *Preprocessing* improves metadata inference by identifying attributes encoded in table content (i.e., values) beyond column headers and sampled rows.

During online table identification, *Joint-Coverage Retrieval and Selection* identifies candidate table sets by maximizing their joint coverage of fine-grained question keyphrases, thereby improving the recall of tables relevant to different parts of the question. *Bridge-Oriented Expansion* then augments selected table sets by prioritizing candidates that can serve as bridge tables to improve overall connectivity, thus pruning spurious candidates. For small collections with fewer than ten tables, following common practice [32, 36], we directly use an LLM to select relevant tables based on the metadata from preprocessing, without applying the full online identification.

3.2 Implementation of Discovery

Preprocessing. Table metadata describes the key attributes represented by a table. However, for non-relational tables, some attributes may not be directly available from the column headers and sampled rows used for metadata inference. Our key observation is that such attributes are often encoded in table content in ways associated with preparation operations. For example, for Transpose, some attributes may appear as values in the first column. We refer to the additional table content associated with the supported operations as operation-oriented evidence.

Thus, for each table $T \in \mathcal{T}$, we first extract operation-oriented evidence using predefined rules over the supported preparation operations. Specifically, we define a rule for each operation to identify its corresponding pattern in the table content and extract the matched content as additional evidence, and implement these rules using existing table profiling tools [1] or LLMs. For structural transformations, we extract the values of first column that behave as attributes for Transpose, and distinct values from columns whose values may become attributes for Pivot. For schema transformations, we extract values that contain multiple fields within a single column for SplitColumn, and use sampled values to infer the meaning of ambiguous column headers for Rename. For value normalizations, which do not change the attributes represented by the table, we extract no additional evidence. Only matched rules contribute additional evidence.

We then provide this evidence, together with the original column headers and sampled rows, to an LLM and prompt it to infer the attributes represented by T, which are used as its metadata.

Joint-Coverage Retrieval and Selection. To avoid missing tables that are relevant to only part of the question, we perform retrieval and selection based on fine-grained question coverage rather than whole-question relevance. Specifically, given a question q and table collection $\mathcal{T}$, we first decompose q into a set of keyphrases K_q using an LLM following [33]. For each keyphrase $k \in K_q$, we compute its relevance $r(k, T)$ to every table $T \in \mathcal{T}$ as the maximum embedding similarity between k and the attributes in the metadata of T. We then rank all tables by their relevance and retain the top-m candidate tables for each keyphrase. After that, we incrementally construct candidate table sets by selecting one candidate table for each keyphrase. To bound the search space, after processing each keyphrase, we retain only the top-b mappings according to their cumulative relevance. Each retained mapping yields a candidate table set C and its corresponding keyphrase–table mapping. For each candidate table set C, we compute its joint question relevance as $\sum_{k \in K_q} r(k, T_k)$, where T_k is the table selected for keyphrase k in the corresponding mapping. This score measures how well the selected tables collectively cover question keyphrases. We rank the candidate table sets by this score and retain the top scoring sets.

Bridge-Oriented Expansion. The retained table sets mainly contain tables directly relevant to the question keyphrases, but may still miss bridge tables needed to connect them. We therefore augment each retained set with intermediate tables that connect its selected tables, rather than greedily adding tables for local compatibility.

Specifically, we organize a graph using the pairwise table compatibility from CORE-T, where each node represents a table and an edge connects a table pair whose compatibility exceeds a predefined threshold. For each retained table set C, we then check whether its selected tables are connected using only the edges among them. If not, we iteratively search for short paths between their disconnected components and add the intermediate tables on the selected paths as candidate bridge tables. These bridge tables augments C to form a candidate expanded table set $\hat{C}$. We then rank all resulting

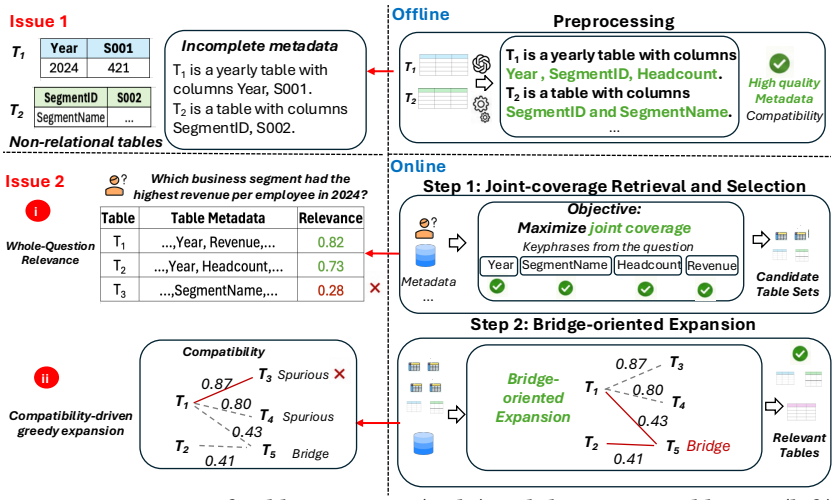


Figure 3: Overview of Table Discovery (right) and the issues it addresses (left), as indicated by red arrows.

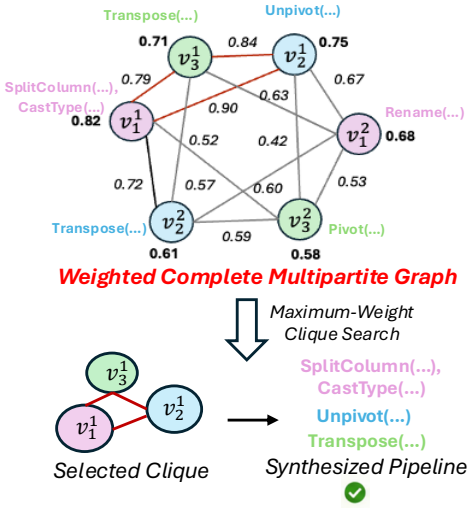


Figure 4: Using weighted complete multipartite graph for pipeline synthesis.

expanded table sets by jointly considering their original question relevance and the connectivity achieved after expansion. For each expanded table set $\hat{C}$ derived from C, we compute its score as $\sum_{k \in K_q} r(k, T_k) + \beta \left(1 - \frac{n_{cc}(\hat{C})-1}{|C|-1}\right)$, where T_k is the table selected for keyphrase k in the mapping associated with C, and $n_{cc}(\hat{C})$ is the number of connected components formed by the tables in $\hat{C}$. The first term measures the joint relevance of the originally selected tables to the question keyphrases. The second term measures the connectivity of the expanded table set and rewards expansions with fewer disconnected components, reaching its maximum when all tables are connected. The parameter β controls the relative contribution of the connectivity term.

Finally, to avoid relying solely on the heuristic relevance and connectivity measures, we retain the top-k expanded table sets according to this score and further rerank them using an LLM following [12]. The highest ranked set is selected as the final relevant table set, and its tables are returned as the relevant tables.

4 CLIQUE-BASED PIPELINE SYNTHESIS

After identifying all relevant tables, the next step is to synthesize a pipeline to prepare each table and required table relationships. This becomes difficult as more preparation steps and table relationships are required. To address it, we formulate pipeline synthesis as a combinatorial search and solve it as a maximum-weight clique problem over a weighted complete multipartite graph (§4.1). We then show how to effectively construct this graph (§4.2).

4.1 From Pipeline to Combinatorial Search

Why Combinatorial Search? During pipeline synthesis, each relevant table can have multiple candidate multi-step preparations. However, these candidates cannot be selected independently, because the choices across tables jointly determine whether the required table relationships can also be properly prepared. Therefore, pipeline synthesis requires searching over combinations of candidate preparations across all relevant tables to satisfy both objectives.

Even in simple cases, the resulting search space is already large. For example, with only 2 relevant tables and 2 preparation steps per table, our 10 preparation operations already yield up to 10^4 possible operation combinations, even before considering operation-specific parameters. This naturally gives rise to a combinatorial search [40]: *given the relevant tables and preparation operation set, find the optimal pipeline by selecting one operation combination for each relevant table from all possible combinations, such that both the tables and their required relationships are correctly prepared.*

Search over a Weighted Complete Multipartite Graph. Because the above search jointly considers the correctness of individual table preparations and their relationships, it naturally admits a graph formulation: candidate operation combinations for each relevant table form one partition, while edges between candidates from different tables capture the correctness of their required relationships. Therefore,we formulate it as a *maximum-weight clique* problem over a *weighted complete multipartite graph* [16, 43].

Concept 1 (Weighted Complete Multipartite Graph). *Given n relevant tables $\{T_1, \ldots, T_n\}$, we define a weighted complete multipartite graph $G = (V, E, w_v, w_e)$, where the vertex set V is partitioned into n disjoint sets $V_1, \ldots, V_n$. Each partition V_i corresponds to a relevant table T_i, and each vertex $v_i^j \in V_i$ represents one executable preparation sequence across multiple steps for preparing T_i. Each vertex is associated with a weight $w_v(v_i^j)$ representing its table preparation correctness. There are no edges between vertices within the same partition, while every pair of vertices from different partitions is connected. Each such edge $(v_i^j, v_k^l) \in E$ is associated with a weight $w_e(v_i^j, v_k^l)$ representing the table relationship correctness between the corresponding prepared results. Edges without a specified table relationship are assigned zero weight.*

Definition 2 (Maximum-Weight Clique). *Given a weighted complete multipartite graph $G = (V, E, w_v, w_e)$ with n partitions*

$V_1, \ldots, V_n$, *the maximum-weight clique problem aims to find an* n*-vertex clique* $C^* \subseteq V$*:*

$$C^* = \arg\max_{C \subseteq V} \left(\sum_{v \in C} w_v(v) + \lambda \sum_{\{u,v\} \subseteq C} w_e(u,v) \right),$$

subject to selecting exactly one vertex from each partition:

$$|C \cap V_i| = 1, \qquad \forall i \in \{1, \ldots, n\}.$$

Here, λ *is a weighting parameter that balances the contributions of vertex and edge weights.*

Example 2. Figure 4 shows the weighted complete multipartite graph for the three relevant tables T_1*–*T_3*, with two representative vertices per partition. Each vertex denotes one candidate preparation sequence for its table (e.g.,* v_3^1 *corresponds to* Transpose *for* T_3*). Selecting one vertex from each partition forms a clique and therefore specifies one preparation pipeline across the three tables.*

Key Technical Challenge. Although the maximum-weight clique problem can be solved using an existing clique search algorithm [43], directly constructing the full graph is impractical because the number of possible multi-step preparation sequences grows exponentially with the number of preparation steps. Accordingly, the key technical challenge is: *how can we effectively prune the vertex space while retaining reliable candidate preparation sequences?*

To address this challenge, we first derive graph construction criteria from the question and use them for *target-guided Vertex Pruning*. We then assign vertex and edge weights.

4.2 Graph Construction

Graph Construction Criteria. Graph construction requires knowing what each relevant table should be prepared as and which relationships should hold among the resulting tables for answering the question. Our key idea is to view this as designing a relational schema [20] conditioned on the question and the identified relevant tables. Accordingly, we provide the question together with the constructed metadata of the relevant tables to an LLM and prompt it to derive a relational schema consisting of a target table schema for each relevant table and the relationships among these tables. Each target table schema specifies the required columns, their data types, and the key columns of the prepared table, while each relationship specifies which prepared tables should be joined and through which columns. This relational schema then serves as the graph construction criteria, providing the required table outputs and cross-table relationships needed to guide graph construction. Since graph construction depends on the synthesized criteria, we further evaluate how their quality affects pipeline synthesis (§6.4).

Target-Guided Vertex Pruning. Each vertex should represent a candidate preparation sequence that transforms a relevant table toward its target table schema. Directly enumerating all possible operation sequences to identify such candidates is expensive, while directly relying on LLMs to generate candidate multi-step sequences becomes unreliable as the number of required preparation steps increases [19, 22]. Our key idea is to decompose such multi-step preparation into *target-guided next-operation prediction*: at each preparation step, we rank candidate operations based on both the current intermediate table and the target table schema, execute the promising ones, and repeat the prediction on the resulting table states. The process terminates once the current table state indicates that no further steps are needed to reach the target schema.

To learn the target-guided prediction, we construct training data from pipelines in the train splits of prior data preparation benchmarks [19], each of which transforms a single raw table toward a specified target table schema through multiple preparation operations. We execute each pipeline step by step and construct one training example before every operation: the input consists of the current intermediate table and the target table schema, while the label is the next preparation operation in the pipeline. After completing the final operation, we construct one additional example from the resulting table with a termination label, indicating that no further preparation is needed. For example, a pipeline Transpose → Rename produces three examples labeled Transpose, Rename, and termination, respectively. After obtaining the training data, we encode each input with a compact feature vector derived from the current intermediate table, the target schema, and the preceding preparation steps following [29], and train a calibrated gradient-boosted decision model [2] to predict the next preparation decision (i.e., a preparation operation or termination).

During inference, at each step, the trained model scores the possible next decisions for the current table state. If termination receives the highest score, the current sequence is completed. Otherwise, to avoid unnecessary search when the model is confident while preserving alternatives when it is uncertain, we retain only the highest-scoring operation when its predicted probability is at least 0.5; otherwise, we retain the top three operations. For each retained operation, we instantiate its operation-specific parameters using an LLM conditioned on the current table state and target table schema, following [14]. The instantiated operations are then executed to obtain new table states, from which operation prediction and parameter instantiation are repeated. The completed sequences are retained as candidate vertices for the corresponding table.

Graph Weighting. We compute the vertex and edge weights by first executing the preparation sequence represented by each candidate vertex. For each vertex v_i^j, executing its sequence on T_i produces a prepared table $\hat{T}_i^j$, which is then evaluated against the target table schema specified by the graph construction criteria:

$$w_v(v_i^j) = s_{\text{col}}(\hat{T}_i^j) + 0.5\, s_{\text{key}}(\hat{T}_i^j) + 0.5\, s_{\text{type}}(\hat{T}_i^j).$$

The column score s_{col} is the fraction of required target columns present in the prepared table. The key score s_{key} is the fraction of distinct non-null values among all non-null values in the key columns. The type score s_{type} is the fraction of produced values that match the data types of their corresponding target columns.

For an edge (v_i^j, v_k^l), we evaluate whether the corresponding prepared tables $\hat{T}_i^j$ and $\hat{T}_k^l$ realize the join relationship specified by the graph construction criteria. Following prior work on value-based joinability [49], we compute the joinability score as:

$$w_e(v_i^j, v_k^l) = s_{\text{join}}(\hat{T}_i^j, \hat{T}_k^l),$$

where s_{join} is the larger of the two containment ratios between the value sets of the specified join columns, measuring how well the values of one column are covered by those of the other. If either specified join column is absent, the edge receives a score of zero.

5 DIAGNOSIS-GUIDED SELF-CORRECTION

The preceding stages identify the relevant tables and synthesize the pipeline to produce the prepared tables and relationships. Since these stages form a dependent workflow, errors introduced in an earlier stage can propagate through subsequent stages and ultimately degrade end-to-end preparation correctness. Motivated by prior work showing that self-correction can improve outputs [23, 34, 41], we investigate how self-correction can also mitigate such error propagation in our setting. In this section, we first introduce the design principles of self-correction for preparation (§5.1), and then present technical mechanisms to realize them (§5.2-§5.3).

5.1 Principles for Self-Correction

We begin with examples to motivate the key design principles.

Example 3. Suppose an irrelevant table is selected in an early stage, which subsequently leads to an incorrect pipeline and final preparation results. To correct the error, we must first revise the selected tables and then rerun the affected downstream stages. If the updated result remains incorrect because of a pipeline error, we must further revise the pipeline and rerun the subsequent execution. In another case, if revising the pipeline does not improve the results, the error may instead originate from an earlier stage, requiring a different correction. Thus, the correction performed in each round depends on the results of previous corrections, making the overall correction process flexible rather than fixed.

Such a correction process also requires correctly identifying the current error, if any. For example, if an error originates from table selection but is mistakenly attributed to pipeline synthesis, repeatedly revising the pipeline cannot correct the actual error and may waste multiple corrections.

Design Principles and Technical Challenges. Thus, we argue that effective self-correction should satisfy two design principles: *support flexible correction across stages and rounds*, and *accurately diagnose the workflow in each round.*

While intuitive, realizing these principles is non-trivial: (1) *How can we support flexible correction across stages and rounds?* Unlike following a fixed correction process, the correction performed in each round may involve different stages and depend on the results of previous corrections, thus requiring a flexible correction structure. (2) *How can we accurately diagnose the workflow in each round?* Each round produces an *execution state*, consisting of the intermediate and final results of one workflow execution, including the identified relevant tables, graph construction criteria, pipeline, prepared tables, and their relationships. However, these results may not directly support diagnosis (i.e., determining whether correction is needed and, if so, which intermediate result is erroneous).

To address these challenges, we first introduce a *correction tree* that organizes the results and corrections across rounds, enabling flexible correction (§5.2). Built on this structure, we further train a *diagnostic policy* using synthesized examples for diagnosis (§5.3).

5.2 Correction Tree

To support flexible corrections across stages and rounds, we organize the correction process as a *correction tree*, where nodes represent execution states, edges represent correction actions, and tree expansion organizes how successive corrections are performed over the explored states.

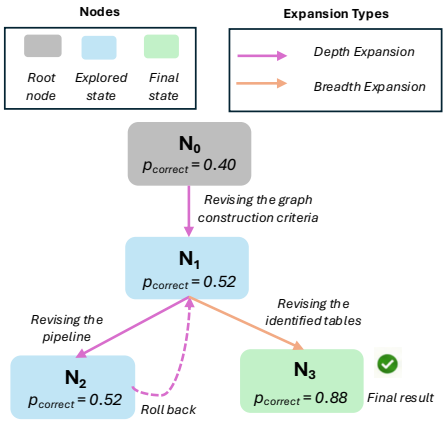


Figure 5: An example of Correction Tree.

Nodes. Each node represents an execution state produced during correction. The root node represents the execution state produced by the initial forward workflow.

Edges. Each edge represents an LLM-based correction action applied to its parent node. We consider three types of correction actions: revising the identified tables, revising the graph construction criteria, and revising the pipeline. Applying an action revises the corresponding intermediate result and re-executes its affected downstream stages to produce the child node.

Tree Expansion. Starting from the root node, we iteratively expand the correction tree. Each expansion requires first diagnosing the current execution state and applying the corresponding correction to produce a child node, and then evaluating the child node to determine where subsequent expansion continues.

For the current node, the diagnostic policy (§5.3) determines whether correction is needed and, if so, which intermediate result is erroneous. If no correction is needed, expansion from the current node stops. Otherwise, the corresponding correction action is applied to the identified erroneous intermediate result, and the affected downstream stages are re-executed to produce a child node.

We then evaluate the produced child node using the same diagnostic policy. If the child node is diagnosed as *correct*, we retain it and stop expansion from this node. Otherwise, we assess whether the correction improves the current state. Let p_{correct} denote the predicted probability that an execution state is correct. Since it reflects the policy's confidence in preparation correctness, we consider a correction effective if the p_{correct} of the child node exceeds that of its parent by more than 0.05 (the threshold selected based on validation performance). Based on this evaluation, subsequent expansion follows one of two directions: *depth expansion* for effective corrections and *breadth expansion* for ineffective corrections. For an effective correction, we retain the child node and continue correction from it in subsequent rounds, as errors may still remain after one correction. For an ineffective correction, we roll back to its parent and consider the highest-probability untried correction action from the same parent.

Termination and Final Result. The correction process terminates when the diagnostic policy determines that no further correction is needed or the correction-round budget is reached. We retain all explored states and finally select the one with the highest p_{correct} as the final result.

Example 4. As shown in Figure 5, the root node N_0 represents the execution state produced by the initial forward workflow, with $p_{\text{correct}} = 0.40$. The diagnostic policy first identifies an error in the graph construction criteria. Revising the criteria and re-executing the downstream stages produces N_1 with $p_{\text{correct}} = 0.52$. Since the improvement is $0.12 > 0.05$, this correction is effective, and the tree performs depth expansion by continuing correction from N_1.
In the next round, the policy identifies a pipeline error in N_1. Revising the pipeline produces N_2 with $p_{\text{correct}} = 0.52$. Since p_{correct} does not improve, this correction is ineffective. The tree therefore rolls back to N_1 and performs breadth expansion by trying the highest-probability untried correction action, which revises the identified relevant tables. This produces N_3 with $p_{\text{correct}} = 0.88$. The policy diagnoses N_3 as correct, so expansion stops. Among all explored states $\{N_0, N_1, N_2, N_3\}$, N_3 has the highest p_{correct} and is returned as the final result.

5.3 Learning the Diagnostic Policy

To support diagnosis, a naive solution is to provide the execution state to an LLM and prompt it to make this diagnosis, similar to [41]. However, the LLM is not explicitly supervised for this diagnosis task, making its predictions unreliable [25] and potentially triggering inappropriate corrections that degrade the preparation results. Moreover, repeatedly invoking the LLM for diagnosis introduces additional inference cost. Therefore, we train a diagnostic policy with explicit supervision for this task. This raises two questions: (1) *How can we construct reliable supervision for training the policy?* (2) *How can we effectively learn the diagnosis from the execution state?*

Training Data Construction. We construct the supervision using queries from train splits of prior data preparation benchmarks [19], together with their ground-truth intermediate and final preparation results. For each query, we execute the forward workflow to obtain the initial execution state and compare its results with their ground-truth counterparts to obtain a diagnostic label. We consider four diagnostic labels: {*correct*, *table error*, *criteria error*, *pipeline error*}. The *correct* label indicates that no correction is needed, while *table error*, *criteria error*, and *pipeline error* indicate errors in the identified relevant tables, graph construction criteria, and pipeline, respectively. If multiple intermediate results are erroneous, we assign the label corresponding to the earliest erroneous result, so that the supervision identifies the upstream error that should be corrected first. Thus, each training example consists of a question, an execution state, and its corresponding diagnostic label.

Learning from Execution State. Given the constructed training examples, we next learn the diagnostic policy from each execution state. Since different diagnostic labels depend on different results in an execution state, using all results indiscriminately may introduce irrelevant signals and hinder accurate diagnosis. We address this through three steps: selecting the results relevant to each diagnostic label, extracting representations from the selected results, and jointly learning the diagnostic labels from these representations.

First, we select the results most relevant to each diagnostic label. For *table error*, we use the identified relevant tables; for *criteria error*, we use the inferred graph construction criteria; and for *pipeline error*, we use the synthesized pipeline. For *correct*, we use the prepared tables and their relationships, together with the column, key, and type scores for each prepared table and their joinability scores (§4.2). These scores provide direct evidence of preparation correctness and thus are included as additional information for *correct*.

Based on the selected results, we then convert them into representations for diagnosis. We first serialize the question and the corresponding results into a unified textual format and encode each serialized question–result pair using a frozen ModernBERT encoder [45]. We choose ModernBERT for its effectiveness and efficiency in text classification tasks. For each encoded pair, we apply mean pooling over the contextualized token embeddings to obtain a single embedding and project it into a 16-dimensional vector.

Finally, we jointly learn the four diagnostic labels from their corresponding representations. For each label, we use a lightweight classification head that maps its corresponding representation, together with additional numerical scores when applicable, to a scalar logit. We then apply a softmax over the four logits to obtain the predicted probabilities over the diagnostic labels. To account for label imbalance [26], we use class-weighted cross-entropy [35] as the training objective, comparing the predicted label probabilities with the ground-truth diagnostic label and assigning larger weights to less frequent labels.

At inference time, the trained diagnostic policy outputs a probability distribution over the four diagnostic labels, where p_{correct} denotes the predicted probability of *correct*. The label with the highest predicted probability is selected as the diagnosis.

6 EXPERIMENTAL EVALUATION

In this section, we answer the following five questions:

- Q1: How effective is PrepWeaver in overall preparation?
- Q2: How robust is PrepWeaver to the three key challenges (§1)?
- Q3: How much does each key component contribute to the effectiveness of PrepWeaver?
- Q4: How efficient is PrepWeaver?
- Q5: How much does PrepWeaver improve downstream question answering over alternative approaches?

6.1 Experimental Setup

Datasets. Evaluating our setting requires benchmarks that provide (1) a question requiring multiple relevant tables, (2) a table collection containing both relevant and irrelevant tables, and (3) ground-truth prepared tables and their relationships.

We first adapt *Synth-Bird and Synth-Spider* [19], two challenging preparation benchmarks with diverse operator types and long operation sequences that are synthesized from Text-to-SQL datasets. We use their test splits for evaluation. As their original setting provides only the relevant tables and target table metadata, we retain the cases with multiple relevant tables while recover our setting from the original Text-to-SQL benchmarks by (i) using the corresponding question as input, (ii) adding irrelevant tables from the same database into the table collection, and (iii) using the original SQL

Property	Synth-Bird	Synth-Spider	BEAVER-Prep
# Questions	141	103	119
Avg. # Input Tables	7.09	4.98	96.00
Avg. # Relevant Tables	2.14	2.31	3.95
Avg. Nonrel. Ratio	0.32	0.29	0.03
Avg. Prep. Steps	2.04	1.41	2.44
Avg. # Relationships	1.16	1.47	3.56

Table 2: Dataset statistics.

annotations to derive the ground-truth prepared tables and the relationships among these prepared tables.

However, their underlying table collections are built on public databases and may not adequately reflect realistic enterprise scenarios [11]. Existing real-world benchmarks such as KramaBench [28] provide end-to-end pipelines over tabular data, but are insufficient for our evaluation because only a small subset of their questions (fewer than 20) involve preparation needs. We therefore construct *BEAVER-Prep* using the enterprise Text-to-SQL benchmark BEAVER [11] and select its DW warehouse split because its table collection lacks explicit relationships. BEAVER provides questions requiring multiple relevant tables, but its original tables are already clean and require no preparation for answering. Thus, we first use the original tables and Text-to-SQL annotations to derive the ground-truth prepared tables and their relationships, and then introduce preparation needs into these clean tables following [19] to construct the input tables. Specifically, we use the preparation statistics observed from the two adapted benchmarks to guide this synthesis. For each table collection, we randomly select 60% of the tables and introduce preparation needs requiring two to three preparation steps for each selected table. At each step, we provide the supported preparation operations as candidates and prompt an LLM to select an appropriate operation and instantiate its operation-specific parameters. We then use selected operation and parameters to modify original table, so that resulting table requires preparation.

Table 2 summarizes the key properties of each dataset, with all averages computed at the question level. Here, *Nonrel. Ratio* denotes the fraction of relevant tables that are non-relational, *# Relationships* denotes the number of corresponding joinable column pairs, and *Prep. Steps* denotes the maximum number of preparation steps required by any relevant table. Since these datasets do not cover a wide range of table collection sizes (see # Input Tables), we do not separately evaluate scalability with respect to collection size.

Baselines. We compare against two groups of baselines. First, we include three recent methods for preparation pipeline synthesis (§2.2): *DeepPrep* [19], *BAT* [21], and *Text-to-Pipeline* [22]. We adapt each method to our question-driven setting by first identifying the relevant tables with CORE-T [42] and then using an LLM to infer the required specifications for guiding pipeline synthesis.

Second, we include methods that answer questions over table collections through executable code generation, where the generated code implicitly performs the data preparation required for answering the question. *GPT-5.5* [39] serves as a direct LLM baseline that performs single-shot code generation over tables retrieved by CORE-T. *Pneuma-Seeker* [8] is an LLM agent that uses relational reification and conductor-style planning to guide data discovery and code generation, while *DS-STAR* [36] is an LLM-powered system that employs sequential planning, plan verification, and refinement to guide multi-step code generation over retrieved data.

Evaluation Metrics. As our task aims to prepare the relevant tables and establish their relationships, we evaluate the effectiveness as follows. Our primary end-to-end metric, *Preparation Correctness*, measures whether all relevant tables are correctly prepared and all required relationships among them are correctly established. It therefore requires correctness in both aspects: (i) *Table Correctness* measures whether the set of produced prepared tables matches the ground-truth prepared tables, following the table matching method in [47]; and (ii) *Relationship Correctness* measures whether the set of predicted relationships (i.e., joinable column pairs) matches the ground-truth relationships. A predicted relationship is considered matched if the values of its joinable column pair match those of the corresponding ground-truth relationship.

For baselines that provide only executable code rather than explicit prepared tables and relationships, we further extract them from the execution traces for evaluation.

Implementation. We use GPT-5 [38] as the backend LLM for PrepWeaver and the baselines whenever an LLM is required. For Table Discovery, we use UAE-Large-V1 [4] as the embedding model for computing relevance following [42]. We retain the top-30 candidate tables for each question keyphrase, set b to 100, compatibility threshold τ to 0.6, connectivity weight β to 1.5, and retain the top-3 expanded sets for final LLM reranking. For Pipeline Synthesis, we set the clique weighting parameter λ to 1.5. For Self-Correction, we set the correction-round budget to 3. All hyperparameters are set following prior work when applicable and otherwise tuned on a small manually sampled held-out subset. The prompts used by PrepWeaver are provided in Appendix B of our technical report [3].

6.2 Overall Effectiveness (Q1)

Table 3 summarizes the overall preparation effectiveness on the three datasets. We observe that: (1) PrepWeaver consistently achieves the highest Preparation Correctness, outperforming the best baseline by 61%, 4%, and 76% on Synth-Bird, Synth-Spider, and BEAVER-Prep, respectively. This demonstrates the effectiveness of PrepWeaver in correctly preparing all relevant tables while establishing their required relationships. (2) PrepWeaver also achieves the highest Table Correctness and Relationship Correctness across all three datasets. This also shows that the higher Preparation Correctness of PrepWeaver comes from both more accurately preparing the relevant tables and more accurately establishing the required relationships among them. (3) BEAVER-Prep is substantially more challenging than the other two benchmarks, with all methods achieving much lower correctness. This shows that preparation is more challenging in the enterprise setting, which involves more relevant tables, preparation steps, and relationships.

6.3 Robustness (Q2)

In this subsection, we evaluate how robust each method is under our primary end-to-end preparation objective for the identified three key challenges. Accordingly, we use *Preparation Correctness* as the main evaluation metric.

Robustness Setup. For the first two challenges, we define rule-based low- and high-complexity groups using their corresponding

Method	Prep. Corr.			Table Corr.			Relation Corr.		
	Synth-Bird	Synth-Spider	BEAVER-Prep	Synth-Bird	Synth-Spider	BEAVER-Prep	Synth-Bird	Synth-Spider	BEAVER-Prep
DeepPrep	0.149	0.388	0.076	0.227	0.592	0.210	0.248	0.456	0.101
BAT	0.234	0.417	0.050	0.376	0.621	0.126	0.440	0.534	0.109
Text-to-Pipeline	0.340	0.670	0.076	0.447	0.748	0.193	0.539	0.767	0.109
GPT-5.5	0.362	0.466	0.059	0.574	0.728	0.210	0.525	0.573	0.092
Pneuma-Seeker	0.262	0.379	0.025	0.291	0.544	0.067	0.511	0.505	0.067
DS-STAR	0.184	0.272	0.059	0.284	0.476	0.168	0.362	0.388	0.076
PrepWeaver	**0.582**	**0.699**	**0.135**	**0.652**	**0.777**	**0.319**	**0.766**	**0.854**	**0.151**

Prep./Table/Relation Corr.: Preparation/Table/Relationship Correctness.

Table 3: Overall effectiveness comparison.

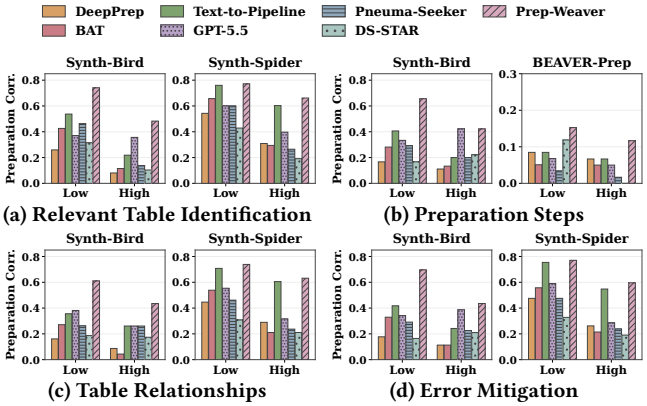


Figure 6: Robustness under the three key challenges.

factors. Specifically, for *Relevant Table Identification*, a question is high complexity if it involves at least three relevant tables or any non-relational relevant table. For *Pipeline Synthesis*, we separately analyze its two complexity factors. A question is considered high complexity in terms of preparation steps if any relevant table requires more than two preparation steps, and high complexity in terms of table relationships if at least two relationships are required for integration. The remaining cases are considered low complexity for the corresponding factor.

Since *Error Mitigation* results from errors in preceding stages propagating through the subsequent workflow and has no standalone complexity factor, we characterize its susceptibility to such error propagation using the combined complexity of these stages. A question is considered high complexity if it is high complexity under any factor of Relevant Table Identification or Pipeline Synthesis defined above; otherwise, it is considered low complexity.

Results. The results are shown in Figure 6. Note that some benchmark results are omitted when either the low- or high-complexity group is unavailable or contains fewer than 10 questions, as such results may be statistically unreliable. We observe that:

(1) *Relevant Table Identification.* As shown in Figure 6(a), most baselines show a noticeable decrease in Preparation Correctness when moving from the low- to high-complexity group, as identifying all relevant tables becomes harder with more relevant tables or non-relational tables. In contrast, PrepWeaver consistently achieves the highest correctness in both groups, showing greater robustness to relevant table identification complexity.

(2) *Pipeline Synthesis.* Figures 6(b)–(c) show that existing methods generally become less effective as more preparation steps or table relationships are required. In contrast, PrepWeaver maintains the highest Preparation Correctness in the high-complexity groups for both factors, showing that it remains effective when pipeline synthesis requires longer steps or more table relationships.

(3) *Error Mitigation.* As shown in Figure 6(d), most baselines exhibit substantial performance degradation in the high-complexity group. In contrast, PrepWeaver remains the best-performing method in the high-complexity group, showing greater end-to-end robustness when the workflow is more susceptible to error propagation.

6.4 Component Effectiveness (Q3)

In this subsection, we evaluate the contribution of the three main components of PrepWeaver: Table Discovery (§3), Pipeline Synthesis (§4), and Self-Correction (§5). Notably, for Table Discovery and Pipeline Synthesis, we evaluate their preparation results before Self-Correction to assess their standalone effectiveness without relying on subsequent correction.

Table Discovery. As Table Discovery is mainly designed to address the limitations of CORE-T, we compare our approach with *CORE-T*. We further analyze the individual design choices of Table Discovery in Appendix A.1 of our technical report [3]. Since the component aims to accurately identify all relevant tables for improving the preparation of these tables and their relationships, we evaluate two aspects: *Table Identification Accuracy*, which measures the percentage of questions whose selected table set exactly matches the ground-truth relevant table set, and the three preparation metrics, which measure the resulting preparation effectiveness.

Table 4 reports the effectiveness of Table Discovery. We observe that: (1) PrepWeaver identifies the relevant tables more reliably than CORE-T across all three datasets. (2) The more accurate relevant table identification also improves preparation on Synth-Bird and Synth-Spider. On BEAVER-Prep, although PrepWeaver improves table identification, Preparation Correctness remains unchanged, indicating that its pipeline synthesis remains a major bottleneck.

Pipeline Synthesis. We evaluate Pipeline Synthesis using the three preparation metrics. We first evaluate its key design for vertex pruning, *target-guided vertex pruning*, by comparing it with *LLM-Vertex*, which directly prompts an LLM to generate multiple candidate operation sequences for each table. We further assess the impact of the synthesized graph construction criteria using *GT-Criteria*, which replaces them with criteria extracted from the ground-truth prepared tables and relationships.

Table 5 reports the effectiveness of Pipeline Synthesis. Observing that: (1) *Target-guided vertex pruning* consistently improves pipeline

Metric	Variant	Synth-Bird	Synth-Spider	BEAVER-Prep
Ident. Acc.[1]	CORE-T	$0.511^{\downarrow 24\%}$	$0.767^{\downarrow 7\%}$	$0.160^{\downarrow 21\%}$
	PrepWeaver	**0.674**	**0.825**	**0.202**
Prep. Corr.	CORE-T	$0.383^{\downarrow 28\%}$	$0.534^{\downarrow 18\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
	PrepWeaver	**0.532**	**0.650**	**0.076**
Table Corr.	CORE-T	$0.518^{\downarrow 12\%}$	$0.621^{\downarrow 12\%}$	$\mathbf{0.269}^{\downarrow 0\%}$
	PrepWeaver	**0.589**	**0.709**	**0.269**
Relation Corr.	CORE-T	$0.589^{\downarrow 14\%}$	$0.660^{\downarrow 19\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
	PrepWeaver	**0.688**	**0.816**	**0.076**

[1] Ident. Acc. denotes Table Identification Accuracy.

Table 4: Effectiveness of Table Discovery. Superscripts indicate relative changes with respect to PrepWeaver.

Metric	Variant	Synth-Bird	Synth-Spider	BEAVER-Prep
Prep. Corr.	LLM-Vertex	$0.390^{\downarrow 27\%}$	$0.602^{\downarrow 7\%}$	$0.067^{\downarrow 11\%}$
	PrepWeaver	**0.532**	**0.650**	**0.076**
	GT-Criteria	0.624	0.621	0.092
Table Corr.	LLM-Vertex	$0.475^{\downarrow 19\%}$	$0.698^{\downarrow 2\%}$	$0.244^{\downarrow 9\%}$
	PrepWeaver	**0.589**	**0.709**	**0.269**
	GT-Criteria	0.738	0.748	0.227
Relation Corr.	LLM-Vertex	$0.546^{\downarrow 21\%}$	$0.709^{\downarrow 13\%}$	$0.067^{\downarrow 11\%}$
	PrepWeaver	**0.688**	**0.816**	**0.076**
	GT-Criteria	0.773	0.767	0.126

Table 5: Effectiveness of Pipeline Synthesis.

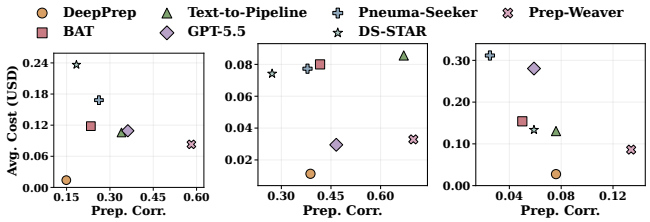
synthesis over direct LLM-based candidate generation. Compared with LLM-Vertex, our design achieves higher Preparation Correctness across all three datasets, with improvements also generally observed in Table and Relationship Correctness. This validates our design of progressively pruning the vertex space based on the target, which produces more reliable candidate preparation sequences than directly generating complete multi-step sequences with an LLM. (2) The impact of graph construction criteria varies across datasets and preparation aspects. GT-Criteria substantially improves Preparation Correctness on Synth-Bird and slightly improves it on BEAVER-Prep. However, on Synth-Spider, it improves Table Correctness but decreases Relationship Correctness. By analyzing these cases, we find that some relationships require little preparation. Since GT-Criteria both provides the ground-truth table schemas and relationships to guide pipeline synthesis, the selected preparation sequences may potentially modify these joinable columns and affect relationship construction.

Self-Correction. To evaluate the contribution of Self-Correction, we first compare with *No-Correction*, which removes this component. We further evaluate its learned diagnostic policy against *LLM-Diagnosis*, which prompts an LLM to perform diagnosis.

Table 6 reports the effectiveness of Self-Correction. We observe that: (1) Compared with *No-Correction*, our Self-Correction consistently improves Preparation Correctness across all three datasets. Similar gains are also observed in both Table Correctness and Relationship Correctness, showing that our correction helps improve correctness in both prepared tables and their relationships. (2) Compared with *LLM-Diagnosis*, our learned diagnostic policy consistently achieves higher Preparation Correctness across all three

Metric	Variant	Synth-Bird	Synth-Spider	BEAVER-Prep
Prep. Corr.	No-Correction	$0.532^{\downarrow 9\%}$	$0.650^{\downarrow 7\%}$	$0.076^{\downarrow 44\%}$
	LLM-Diagnosis	$0.553^{\downarrow 5\%}$	$0.592^{\downarrow 15\%}$	$0.126^{\downarrow 6\%}$
	PrepWeaver	**0.582**	**0.699**	**0.135**
Table Corr.	No-Correction	$0.589^{\downarrow 10\%}$	$0.709^{\downarrow 9\%}$	$0.269^{\downarrow 16\%}$
	LLM-Diagnosis	$0.638^{\downarrow 2\%}$	$0.718^{\downarrow 8\%}$	$0.303^{\downarrow 5\%}$
	PrepWeaver	**0.653**	**0.777**	**0.319**
Relation Corr.	No-Correction	$0.688^{\downarrow 10\%}$	$0.816^{\downarrow 5\%}$	$0.076^{\downarrow 50\%}$
	LLM-Diagnosis	$0.723^{\downarrow 6\%}$	$0.728^{\downarrow 15\%}$	$0.135^{\downarrow 11\%}$
	PrepWeaver	**0.766**	**0.854**	**0.151**

Table 6: Effectiveness of Self-Correction.



(a) Synth-Bird. (b) Synth-Spider. (c) BEAVER-Prep.

Figure 7: Preparation effectiveness versus monetary cost.

datasets, with corresponding gains in Table Correctness and Relationship Correctness. This shows that our learned policy provides more reliable diagnosis for guiding the correction process than directly relying on an LLM. (3) *LLM-Diagnosis* can underperform *No-Correction*, as on Synth-Spider, due to unreliable diagnosis.

6.5 Efficiency (Q4)

Since all compared methods rely on LLMs, we evaluate their efficiency using three complementary metrics following [7]: *monetary cost*, measured by the average LLM inference cost per question; *token consumption*, measured by the average numbers of input and output tokens per question; and *LLM inference time*, measured by the average time spent on LLM inference per question.

6.5.1 Cost Efficiency. Figure 7 compares the average monetary cost per question with Preparation Correctness. Methods closer to the lower-right corner achieve a better effectiveness–cost trade-off. We observe that: (1) PrepWeaver achieves the best effectiveness–cost trade-off across all three datasets, with the highest Preparation Correctness and among the lowest monetary costs. Although DeepPrep has the lowest monetary cost, this is largely because its pipeline synthesis uses a specially trained open-source model, and only its table identification incurs LLM API cost. (2) Across datasets, BEAVER-Prep generally incurs the highest monetary cost. This is because it involves a substantially larger table collection, more relevant tables, and more preparation steps and relationships.

6.5.2 Token Efficiency. Table 7 reports the average input and output token consumption per question, together with a component-level breakdown for PrepWeaver. Observing that: (1) Compared with the baselines, PrepWeaver maintains relatively low token consumption across all three datasets, with relatively low input-token

Method	Synth-Bird		Synth-Spider		BEAVER-Prep	
	InTok[1]	OutTok	InTok	OutTok	InTok	OutTok
DeepPrep	23,724	2,327	27,206	2,806	45,360	5,092
BAT	37,791	3,712	20,886	3,102	49,368	4,832
Text-to-Pipeline	29,167	3,916	20,734	3,521	33,434	5,150
GPT-5.5	16,020	971	2,679	536	31,047	4,171
Pneuma-Seeker	28,188	8,492	30,068	1,761	133,478	5,571
DS-STAR	58,389	9,596	13,141	3,658	27,557	6,090
PrepWeaver	**17,066**	**3,795**	**7,645**	**1,396**	**27,645**	**2,702**
Table Discovery	559	183	459	200	5,369	809
Pipeline Synthesis	11,458	1,052	6,342	648	19,416	1,700
Self-Correction	5,049	2,560	844	547	2,860	193

[1] InTok/OutTok denote the average numbers of input/output tokens per question.

Table 7: LLM token consumption and component breakdown of PrepWeaver.

Method	Synth-Bird	Synth-Spider	BEAVER-Prep
DeepPrep	37.7	56.5	140.3
BAT	79.5	92.5	104.2
Text-to-Pipeline	43.1	38.6	61.5
GPT-5.5	14.6	10.0	65.5
Pneuma-Seeker	21.8	28.0	98.9
DS-STAR	65.7	41.3	62.4
PrepWeaver	**55.7**	**35.9**	**113.3**
Table Discovery	3.6	3.9	11.5
Pipeline Synthesis	32.8	27.5	81.5
Self-Correction	19.3	4.5	20.3

Table 8: LLM inference time (seconds).

usage and moderate output-token usage. This indicates that PrepWeaver is token-efficient compared with the alternative approaches. (2) Across datasets, BEAVER-Prep generally requires the most tokens, while Synth-Spider requires fewer. This demonstrates that larger table collections, more relevant tables, and more preparation steps and relationships can lead to higher token consumption. (3) Within PrepWeaver, Pipeline Synthesis consumes the most tokens across all three datasets. This is because Pipeline Synthesis repeatedly generates and instantiates operations across multiple candidate sequences. Table Discovery also consumes substantially more tokens on BEAVER-Prep due to its much larger input table collection, while the token consumption of Self-Correction varies with the number of correction rounds invoked.

6.5.3 Time Efficiency. Table 8 reports the average LLM inference time per question, together with a component-level breakdown for PrepWeaver. We observe that: (1) Compared with the baselines, PrepWeaver incurs moderate LLM inference time on Synth-Bird and Synth-Spider, while requiring more time on BEAVER-Prep. This is because BEAVER-Prep involves a larger table collection, more relevant tables, and more preparation steps and relationships, which require more LLM calls during preparation. (2) For PrepWeaver, Pipeline Synthesis dominates the LLM inference time across all three datasets, accounting for 59%, 77%, and 72% of the total time on Synth-Bird, Synth-Spider, and BEAVER-Prep, respectively. This

Method	Synth-Bird	Synth-Spider	BEAVER-Prep
DeepPrep	0.106	0.311	0.008
BAT	0.206	0.359	0.008
Text-to-Pipeline	0.241	0.456	0.008
Pneuma-Seeker	0.227	0.311	0.008
DS-STAR	0.326	0.398	**0.034**
PrepWeaver	**0.440**	**0.524**	**0.034**
GPT-5.5	0.418	0.621	0.034

Table 9: Comparison of downstream answer accuracy.

is because Pipeline Synthesis repeatedly instantiates operation-specific parameters for candidate sequences using the LLM.

6.6 Downstream Utility (Q5)

In this subsection, we evaluate downstream utility by measuring how well the prepared tables and relationships support answering.

6.6.1 Evaluation Setup. We use the same three preparation benchmarks. For each question, we obtain the ground-truth answer by executing its original gold SQL from corresponding Text-to-SQL benchmark. We evaluate downstream question answering performance using *Answer Accuracy*, defined as the percentage of questions for which the predicted answer matches ground-truth answer.

For PrepWeaver, DeepPrep, BAT, and Text-to-Pipeline, we provide their prepared tables and relationships to the GPT-5 model [38] and prompt it to generate an SQL query for answering the question based on the schemas of the prepared tables and their relationships. We then execute the generated SQL to obtain the predicted answer. For Pneuma-Seeker, DS-STAR, and GPT-5.5, which directly generate executable code, we use the execution result of their generated code as the predicted answer.

6.6.2 Results. Table 9 reports the downstream Answer Accuracy. PrepWeaver achieves the highest accuracy on Synth-Bird and remains competitive on Synth-Spider and BEAVER-Prep. This shows that the more accurate prepared tables and relationships produced by PrepWeaver better support downstream question answering. On Synth-Spider, although GPT-5.5 performs better because PrepWeaver uses GPT-5 for downstream answering, replacing it with GPT-5.5 increases PrepWeaver's accuracy to 0.641, exceeding the direct GPT-5.5 baseline of 0.621.

7 CONCLUSION

In this paper, we propose PrepWeaver, a self-correcting preparation framework for Question-Driven Tabular Data Preparation. PrepWeaver consists of: Table Discovery, which improves offline metadata and identifies relevant tables through joint question coverage and bridge-table expansion; Clique-Based Pipeline Synthesis, which synthesizes the preparation pipeline through maximum-weight clique search over a weighted complete multipartite graph; and Diagnosis-Guided Self-Correction, which uses a learned diagnostic policy and a correction tree to mitigate error propagation in the workflow. Experiments on three benchmarks show that PrepWeaver consistently improves Preparation Correctness while achieving a strong effectiveness–cost trade-off.

REFERENCES

[1] [n. d.]. DataPrep library. https://docs.dataprep.ai/user_guide/user_guide.html.

[2] [n. d.]. Gradient-boosted Classifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html.

[3] [n. d.]. Technical Report. https://github.com/JrJessyLuo/Prep-Weaver/blob/main/technical_report.pdf.

[4] 2024. WhereIsAI/UAE-Large-V1. https://huggingface.co/WhereIsAI/UAE-Large-V1.

[5] 2025. New Research Reveals that AI Brings Productivity Gains, but Reliance on Spreadsheets Puts Data Quality at Risk. https://www.alteryx.com/about-us/newsroom/press-release/new-research-reveals-that-ai-brings-productivity-gains-but-reliance-on-spreadsheets-puts-data-quality-at-risk.

[6] Rishita Agarwal, Himanshu Singhal, Peter Baile Chen, Manan Roy Choudhury, Dan Roth, and Vivek Gupta. 2025. REaR: Retrieve, Expand and Refine for Effective Multitable Retrieval. *arXiv preprint arXiv:2511.00805* (2025).

[7] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. 2026. How do ai agents spend your money? analyzing and predicting token consumption in agentic coding tasks. *arXiv preprint arXiv:2604.22750* (2026).

[8] Muhammad Imam Luthfi Balaka, John Hillesland, Kemal Badur, and Raul Castro Fernandez. 2026. Pneuma-Seeker: A Relational Reification Mechanism to Align AI Agents with Human Work over Relational Data. *arXiv preprint arXiv:2603.10747* (2026).

[9] Allaa Boutaleb, Bernd Amann, Rafael Angarita, and Hubert Naacke. [n. d.]. Exploring Multi-Table Retrieval Through Iterative Search. In *EurIPS 2025 Workshop: AI for Tabular Data*.

[10] Mengshi Chen, Yuxiang Sun, Tengchao Li, Jianwei Wang, Kai Wang, Xuemin Lin, Ying Zhang, and Wenjie Zhang. 2025. Empowering Tabular Data Preparation with Language Models: Why and How? *arXiv preprint arXiv:2508.01556* (2025).

[11] Peter Baile Chen, Fabian Wenz, Yi Zhang, Devin Yang, Justin Choi, Nesime Tatbul, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2024. BEAVER: an enterprise benchmark for text-to-sql. *arXiv preprint arXiv:2409.02038* (2024).

[12] Peter Baile Chen, Yi Zhang, Mike Cafarella, and Dan Roth. 2025. Can we retrieve everything all at once? ARM: An alignment-oriented LLM-based retrieval method. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 30298–30317.

[13] Peter Baile Chen, Yi Zhang, and Dan Roth. 2024. Is table retrieval a solved problem? exploring join-aware multi-table retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2687–2699.

[14] Wei-Hao Chen, Weixi Tong, Amanda Case, and Tianyi Zhang. 2025. Dango: a mixed-initiative data wrangling system using large language model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–28.

[15] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao 'Kenneth' Huang, Bryan R. Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics, 3697–3711.

[16] Milind Dawande, Pinar Keskinocak, Jayashankar M Swaminathan, and Sridhar Tayur. 2001. On bipartite and multipartite clique problems. *Journal of Algorithms* 41, 2 (2001), 388–403.

[17] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proceedings of the VLDB Endowment* 16, 10 (2023), 2458–2470.

[18] Meihao Fan, Ju Fan, Nan Tang, Lei Cao, Guoliang Li, and Xiaoyong Du. 2025. AutoPrep: Natural Language Question-Aware Data Preparation with a Multi-Agent Framework. *Proc. VLDB Endow.* 18, 10 (2025), 3504–3517.

[19] Meihao Fan, Ju Fan, Yuxin Zhang, Shaolei Zhang, Xiaoyong Du, Jie Song, Peng Li, Fuxin Jiang, Tieying Zhang, and Jianjun Chen. 2026. DeepPrep: An LLM-Powered Agentic System for Autonomous Data Preparation. *arXiv preprint arXiv:2602.07371* (2026).

[20] Wolfgang Gatterbauer and Cody Dunne. 2025. Relational Diagrams and the Pattern Expressiveness of Relational Languages. *ACM SIGMOD Record* 54, 1 (2025), 80–88.

[21] Congcong Ge, Yachuan Liu, Yixuan Tang, Yifan Zhu, Yaofeng Tu, and Yunjun Gao. 2026. BAT: Target-Instance-Free Data Preparation Synthesis via LLM-Driven Tree Search. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD (2026), 1–25.

[22] Yuhang Ge, Yachuan Liu, Zhangyan Ye, Yuren Mao, and Yunjun Gao. 2025. Text-to-pipeline: Bridging natural language and data preparation pipelines. *arXiv preprint arXiv:2505.15874* (2025).

[23] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, Vol. 2024. 57734–57811.

[24] Xuming Hu, Chuan Lei, Xiao Qin, Asterios Katsifodimos, Christos Faloutsos, and Huzefa Rangwala. 2025. POLYJOIN: Semantic Multi-key Joinable Table Search in Data Lakes. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 384–395.

[25] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.

[26] Justin M Johnson and Taghi M Khoshgoftaar. 2019. Survey on deep learning with class imbalance. *Journal of big data* 6, 1 (2019), 27.

[27] Christos Koutras, Jiani Zhang, Xiao Qin, Chuan Lei, Vassilis N. Ioannidis, Christos Faloutsos, George Karypis, and Asterios Katsifodimos. 2025. OmniMatch: Joinability Discovery in Data Products. *Proc. VLDB Endow.* 18, 11 (2025), 4588–4601.

[28] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Om Chabra, Sivaprasad Sudhir, Anna Zeng, Anton A Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, et al. 2025. Kramabench: A benchmark for ai systems on data-to-insight pipelines over data lakes. *arXiv preprint arXiv:2506.06541* (2025).

[29] Eugenie Y Lai, Yeye He, and Surajit Chaudhuri. 2025. Auto-Prep: Holistic Prediction of Data Preparation Steps for Self-Service Business Intelligence. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2212–2225.

[30] Fengyu Li, Junhao Zhu, Kaishi Song, Lu Chen, Zhongming Yao, Tianyi Li, and Christian S. Jensen. 2026. Replacing Multi-Step Assembly of Data Preparation Pipelines with One-Step LLM Pipeline Generation for Table QA. *Proc. VLDB Endow.* 19, 9 (2026), 2372–2384.

[31] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chauduri. 2023. Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples. *arXiv preprint arXiv:2307.14565* (2023).

[32] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2025. A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering* (2025).

[33] Feng Luo, Hai Lan, Hui Luo, Zhifeng Bao, Xiaoli Wang, J. Shane Culpepper, and Shazia Sadiq. 2026. Decomposition-Driven Multi-Table Retrieval and Reasoning for Numerical Question Answering. In *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, QC, Canada, May 4-8, 2026*. IEEE, 1872–1885.

[34] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.

[35] Anqi Mao, Mehryar Mohri, and Yutao Zhong. 2023. Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning*. pmlr, 23803–23828.

[36] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, and Tomas Pfister. 2025. DS-STAR: Data Science Agent via Iterative Planning and Verification. *arXiv preprint arXiv:2509.21825* (2025).

[37] Arash Dargahi Nobari and Davood Rafiei. 2025. TabulaX: Leveraging Large Language Models for Multi-Class Table Transformations. *Proc. VLDB Endow.* 18, 11 (2025), 3826–3839.

[38] OpenAI. 2025. GPT-5. https://platform.openai.com/docs/models/gpt-5.

[39] OpenAI. 2026. GPT-5.5. https://platform.openai.com/docs/models/gpt-5.5.

[40] Christos H Papadimitriou and Kenneth Steiglitz. 1998. *Combinatorial optimization: algorithms and complexity*. Courier Corporation.

[41] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[42] Hassan Soliman, Vivek Gupta, Dan Roth, and Iryna Gurevych. 2026. CORE-T: COherent REtrieval of Tables for Text-to-SQL. *arXiv preprint arXiv:2601.13111* (2026).

[43] Yiyuan Wang, Shaowei Cai, and Minghao Yin. 2016. Two efficient local search algorithms for maximum weight clique problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30.

[44] Yuhui Wang, Jinqi Liu, Chengliang Chai, Hangyu Zhao, Yuhao Deng, Yuyu Luo, Xin Tang, Ye Yuan, Guoren Wang, Fengjin Wang, et al. 2026. EcoTable: Cost-effective Table Integration in Data Lakes for Natural Language Queries. *arXiv preprint arXiv:2606.26613* (2026).

[45] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, et al. 2025. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 2526–2547.

[46] Jian Wu, Linyi Yang, Dongyuan Li, Yuliang Ji, Manabu Okumura, and Yue Zhang. 2025. MMQA: Evaluating LLMs with multi-table multi-hop complex questions. In *The Thirteenth International Conference on Learning Representations*. 1.

[47] Jingzhe Xu, Rui Wang, Jiannan Wang, and Guoliang Li. 2026. PrepBench: How Far Are We from Natural-Language-Driven Data Preparation? *arXiv preprint*

arXiv:2605.08687 (2026).

[48] Jiani Zhang, Sercan O Arik, Cosmin Arad, Fatma Ozcan, and Alon Halevy. 2026. An Agentic Approach to Metadata Reasoning. *arXiv preprint arXiv:2604.20144* (2026).

[49] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J Miller. 2019. Josie: Overlap set similarity search for finding joinable tables in data lakes. In *Proceedings of the 2019 International Conference on Management of Data*. 847–864.

[50] Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, et al. 2025. A Survey of Data Agents: Emerging Paradigm or Overstated Hype? *arXiv preprint arXiv:2510.23587* (2025).

A EXPERIMENTAL DETAILS

A.1 Table Discovery

To evaluate the individual design choices of Table Discovery, we consider three variants. For *Preprocessing*, we compare with *No-Evidence*, which removes operation-oriented evidence and infers table metadata using only the column headers and sampled rows. For online identification, which is applied to large table collections, we evaluate the following two design choices on BEAVER-Prep. For *Joint-Coverage Retrieval and Selection*, we compare with *Whole-Question*, which replaces fine-grained question coverage with whole-question relevance and selects the highest-ranked tables to form a table set of the same size. For *Bridge-Oriented Expansion*, we compare with *Greedy Expansion*, which greedily adds tables based on local pairwise compatibility.

Results. The results are shown in Table 10. We observe that: (1) *Preprocessing* consistently improves Table Identification Accuracy across all three datasets and also improves Table Correctness. On Synth-Bird and Synth-Spider, these gains further translate into higher Preparation and Relationship Correctness. This shows that operation-oriented evidence improves metadata quality, thereby benefiting table identification and downstream preparation. (2) For online identification on BEAVER-Prep, both *Joint-Coverage Retrieval and Selection* and *Bridge-Oriented Expansion* substantially improve Table Identification Accuracy and also increase Table Correctness. However, Preparation and Relationship Correctness remain unchanged because subsequent pipeline synthesis remains highly challenging on BEAVER-Prep. We further examine whether potentially less reliable table–table compatibility for non-relational tables limits the benefit of Bridge-Oriented Expansion. Compared with Greedy Expansion, Bridge-Oriented Expansion improves Table Identification Accuracy by 29% on non-relational cases, which is even higher than the 24% improvement on all-relational cases. This suggests that Bridge-Oriented Expansion is relatively robust to imprecise table–table compatibility because it relies less on exact compatibility scores.

(a) Preprocessing

Metric	Variant	Synth-Bird	Synth-Spider	BEAVER-Prep
Ident. Acc.[1]	No-Evidence	$0.511^{\downarrow 24\%}$	$0.767^{\downarrow 7\%}$	$0.185^{\downarrow 8\%}$
	PrepWeaver	**0.674**	**0.825**	**0.202**
Prep. Corr.	No-Evidence	$0.383^{\downarrow 28\%}$	$0.534^{\downarrow 18\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
	PrepWeaver	**0.532**	**0.650**	**0.076**
Table Corr.	No-Evidence	$0.518^{\downarrow 12\%}$	$0.621^{\downarrow 12\%}$	$0.244^{\downarrow 9\%}$
	PrepWeaver	**0.589**	**0.709**	**0.269**
Relation Corr.	No-Evidence	$0.589^{\downarrow 14\%}$	$0.660^{\downarrow 19\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
	PrepWeaver	**0.688**	**0.816**	**0.076**

(b) Online Identification on BEAVER-Prep

Variant	Ident. Acc.	Table Corr.	Prep. Corr.	Relation Corr.
Whole-Question	$0.118^{\downarrow 42\%}$	$0.235^{\downarrow 12\%}$	$\mathbf{0.076}^{\downarrow 0\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
Greedy Expansion	$0.151^{\downarrow 25\%}$	$0.118^{\downarrow 56\%}$	$\mathbf{0.076}^{\downarrow 0\%}$	$\mathbf{0.076}^{\downarrow 0\%}$
PrepWeaver	**0.202**	**0.269**	**0.076**	**0.076**

Table 10: Ablation study of Table Discovery.

B PROMPTS

The following lists all the prompts in PrepWeaver, including the Operation-Oriented Evidence Identification Prompt, Metadata Inference Prompt, and Relevant Table Selection Prompt (§3.2); Relational Schema Derivation Prompt and Operation Parameter Instantiation Prompt (§4.2); Relevant Table Revision Prompt, Graph Construction Criteria Revision Prompt, and Pipeline Revision Prompt (§5.2).

Operation-Oriented Evidence Identification Prompt

You are identifying operation-oriented evidence for table metadata discovery.
Given a raw table preview, inspect whether table metadata may be hidden beyond ordinary column headers.

- **Goal**: Identify preparation-operation patterns that may change the attributes represented by the table.
- **Supported Issue Types**:
  - Transpose: row values behave as attribute names.
  - Pivot: one column stores latent attribute names and another stores values.
  - Unpivot: sibling columns represent values of the same variable.
  - SplitColumn: one cell contains multiple semantic fields.
  - Concatenate: multiple columns jointly express one attribute.
  - Rename: headers are generic, abbreviated, cryptic, or misleading.
- **Rules**:
  - Be conservative and do not invent attributes.
  - Only matched rules contribute evidence.
  - If the table appears relational, return an empty evidence list.
  - Evidence should quote representative headers, row values, column values, or cells.
- **Output JSON**:

```
{
  "operation_evidence": [
    {
      "issue_type": "...",
      "operation": "...",
      "matched": true,
      "evidence": [...],
      "rationale": "..."
    }
  ]
}
```

Metadata Inference Prompt

You are recovering linking-oriented metadata attributes for a raw table.
Given the original column headers, sampled rows, and operation-oriented evidence, infer the semantic attributes represented by the table. These attributes are used as metadata for downstream schema linking.

- **Input**:
  - original column headers;
  - sampled rows from the raw table;
  - operation-oriented evidence from the previous issue-identification step.
- **Goal**: Recover all metadata attributes represented by the table, including attributes hidden in row values, column values, sibling headers, or composite cells.
- **Rules**:
  - Use operation-oriented evidence as grounding.
  - Preserve original headers when they are already meaningful.
  - Do not add attributes unsupported by headers, sampled rows, or evidence.
- **Output JSON**:

```
{
  "inferred_metadata": ["attribute_1", "attribute_2", "..."],
  "metadata_rationale": "..."
}
```

Question Decomposition Prompt

You are decomposing a natural-language question into non-composite noun concepts and attributes for table discovery.
Given a question, decompose it into a series of faithful noun concepts and attributes. If the concepts are uncertain, output only attributes. Each concept or attribute must be wrapped in <sub_c> and </sub_c> tags. After all required concepts are produced, output <FIN></FIN>.

- **Rules**:
  - Preserve every modifier or qualifier in the question. Do not generalize phrases such as "HR department name", "gross square footage", or "assignable area".
  - Use the form `entity:attribute` only when the entity that owns the attribute is explicitly named in the question.
  - Do not prepend an entity that is merely the queried subject, container, or context.
  - Never convert a mentioned entity into an aggregate or count. Counts, totals, numbers of items, and averages are operations, not concepts.
  - When the question already contains a clean noun phrase, output that noun phrase faithfully instead of re-bucketing it under an unrelated entity.
- **Output Format**:

```
<sub_c>concept_or_attribute</sub_c>
<sub_c>entity:attribute</sub_c>
<FIN></FIN>
```

Examples

```
Question:
For movies with the keyword of "civil war", calculate the average revenue
generated by these movies.

Output:
<sub_c>movies:keyword</sub_c>
<sub_c>movies:revenue</sub_c>
<FIN></FIN>
```

```
Question:
List the building names and the HR department name occupying them, and the
number of organizations per building.

Output:
<sub_c>building:name</sub_c>
<sub_c>HR department name</sub_c>
<sub_c>organizations</sub_c>
<FIN></FIN>
```

```
Question:
For each room, show its organization name, the floor number, and the total
square footage.

Output:
<sub_c>organization:name</sub_c>
<sub_c>floor number</sub_c>
<sub_c>square footage</sub_c>
<FIN></FIN>
```

Relevant Table Selection Prompt

You are identifying the relevant tables for question-driven table preparation.
Given a user question and a set of candidate input table files, select the minimal set of table files needed to construct prepared tables that can be integrated to answer the question.
Use each candidate table's inferred metadata as the main signal for the attributes represented by the table. Use original headers and sampled rows as supporting evidence, especially when an identifier, code, date, or measure is ambiguous. Select tables based on the full question, not only on exact word matches.

- **Input**: question, candidate table files, inferred metadata, original headers, and sampled rows.
- **Rules**:
  - Use only the provided candidate table files.
  - Select the minimal set of relevant table files needed to answer the question.
  - The selected tables should collectively preserve the entities, measures, filters, and join keys required by the question.
  - Include bridge tables only when they are needed to connect relevant tables.
  - Do not select a table merely because it shares a column name with another selected table.
  - Prefer inferred metadata over raw headers when the two disagree.
  - If inferred metadata is incomplete, use original headers and sampled rows to recover likely identifiers or measures.
  - Return file names exactly as shown in the candidate list.
  - Do not include subquestions.
- **Output JSON**:

```
{
  "relevant_table_files": ["..."],
  "table_matches": [
    {
      "table_file": "...",
      "evidence": [
        "short reason based on metadata, headers, or sampled rows"
      ]
    }
  ]
}
```

Relational Schema Derivation Prompt

You derive a relational schema for question-conditioned table preparation.
Given a natural-language question and the constructed metadata of the relevant raw tables, derive the prepared schemas and relationships needed to answer the question.

- **Input**: question and relevant-table metadata, including logical table id, source table, original headers, inferred metadata, sampled rows, and optional operation evidence.
- **Goal**:
  - Emit a full target schema for each relevant table after preparation.
  - Emit equality relationships among prepared tables.
- **Rules**:
  - Use only the provided relevant tables.
  - Treat `inferred_metadata` as the main grounding signal.
  - Use original headers and sampled rows to infer types, keys, and relationship columns.
  - Use operation evidence only as provenance; do not re-detect operations.
  - Emit each table's full prepared schema, not only columns mentioned in the question.
  - Include columns needed to answer the question and keys needed to integrate tables.
  - Relationship columns must appear in the corresponding prepared schema.
  - Do not invent relationships that are not needed.
- **Output JSON**:

```
{
  "gold_tables": [
    {
      "logical_table": "table_1",
      "input_file": "...",
      "db_table": "...",
      "create_table_sql": "CREATE TABLE ...",
      "metadata_rationale": "..."
    }
  ],
  "join_edges": [
    {
      "left_table": "table_1",
      "left_on": "...",
      "right_table": "table_2",
      "right_on": "...",
      "rationale": "..."
    }
  ]
}
```

Operation Parameter Instantiation Prompt

You are generating parameter candidates for exactly one data-preparation operation. The operation type is fixed. Do not choose another operation.

- **Input**: fixed operation type, retrieved solved example, current table columns and preview, target relation schema, missing target columns, joinability targets, and the active join target.
- **Goal**: Generate parameter candidates that both materialize the target relation and make the active join edge equi-joinable.
- **Rules**:
  - Return three different parameterizations when possible.
  - The operation name in every candidate must exactly match the fixed operation.
  - Use only source columns visible in the current table.
  - Use target-schema names for newly created or renamed columns.
  - Prefer parameters that expose or repair the active join key.
  - Do not optimize unrelated columns when the active join key is missing or has low overlap.
  - For function-based operations, include a robust Python function string.
- **Output JSON**:

```
{
  "candidates": [
    {"op": "FixedOperation", "params": {...}},
    {"op": "FixedOperation", "params": {...}}
  ]
}
```

Relevant Table Revision Prompt

You are revising relevant-table identification for question-conditioned table preparation.
A previous relevant-table selection was judged incorrect. Select again using the diagnostics below.

- **Diagnostics May Include**:
  - previous table identification: previously selected relevant tables;
  - relational schema: target schemas and relationships derived for the previous selection;
  - pipeline candidates/chains: operation sequences explored for selected tables;
  - prepared tables: materialized outputs produced by those chains;
  - source tables: raw candidate tables and previews/evidence;
  - failure diagnosis: failed requirements, values, columns, or relationships.
- **Rule**: Use diagnostics only to revise relevant-table identification.
- **Goal**: Select the source tables needed to prepare and integrate data for answering the question.

Graph Construction Criteria Revision Prompt

You are revising a relational schema for question-conditioned table preparation.
The relational schema declares prepared-table schemas and relationships. Pipeline synthesis materializes this schema, but the result does not answer the question. Decide whether the relational schema is at fault and, if so, revise only faulty target schemas and/or relationships.

- **Input**: question, current relational schema, source-table contents, produced prepared tables, join-edge health, unmet declared columns, buildable pipeline outputs, and relational-schema repair hints.
- **Test**: For each requirement implied by the question, check whether some declared prepared column or relationship can carry the required values in the needed form.
- **Do Not Change**:
  - table numbering or logical table ids;
  - harmless names when values are already correct;
  - primary keys unless they block materialization or relationships;
  - extra source columns merely because they exist.
- **Output JSON**:

```
{
  "schema_at_fault": true,
  "unmet_requirement": "...",
  "reasoning": "...",
  "revised_tables": [
    {"input_file": "...", "columns": [...], "primary_key": [...]}
  ],
  "revised_join_edges": [
    {
      "left_table": "...",
      "left_on": "...",
      "right_table": "...",
      "right_on": "..."
    }
  ]
}
```

Pipeline Revision Prompt

You are a pipeline repair agent for question-conditioned table preparation.
A data-preparation pipeline produced incorrect prepared tables. First identify which prepared-table output is at fault, then either accept an existing candidate or write a pandas transform to repair it.

- **Input**: question, produced prepared tables, declared join edges, candidate pipelines and outputs, raw source tables, structural evidence, actionable repair hints, and a failure summary.
- **Actionable Repair Hints**: deterministic clues about missing value carriers, wrong key columns, or unfinished transformations.
- **Failure Summary**: execution/scoring diagnosis such as missing columns, wrong value domains, low join-key overlap, or answer mismatch.
- **Rules**:
  - Judge from values, not only column names.
  - Accept an existing candidate if it already produces the required values.
  - Otherwise repair the closest candidate; start from raw table only if needed.
  - Preserve useful columns and join keys.
  - The repair code must define `transform(df)` and return a DataFrame.
- **Output JSON**:

```
{
  "at_fault": ["table_1"],
  "reason": "...",
  "what_is_missing": "...",
  "accept_candidate": null,
  "closest_candidate": null,
  "start_from": "candidate",
  "code": "def transform(df):\n    ...\n    return df"
}
```